Paesano: Holonomic Autonomous Mobile Robotics Platform

Architecture, Estimation, and Control

Draft Report, Work in Progress

Joseph Marra

April 7, 2026

Contents

1 Introduction **3**

2 Hardware Architecture **3**
2.1 Parts List . . . 3
2.2 System Overview . . . 4
2.3 Power Architecture . . . 4
2.4 Motor Control Stack . . . 5
2.5 Sensor Suite . . . 5
2.6 Mechanical Design . . . 6
2.7 Design Philosophy . . . 6

3 Firmware **7**
3.1 Communication Protocol . . . 7
3.2 Control Loop Architecture . . . 8
3.3 Motor Control Algorithms . . . 8
3.3.1 Velocity PID Control . . . 8
3.3.2 Position PID Control . . . 8

4 Software Architecture **9**

5 Kinematics **10**

6 Odometry **10**

7 State Estimation **11**
7.1 Extended Kalman Filter (EKF) . . . 11
7.1.1 Purpose in the System . . . 12
7.1.2 State Vector . . . 12
7.1.3 Covariance Tuning . . . 12
7.2 Monte Carlo Localization (MCL) . . . 13
7.2.1 Purpose in the System . . . 13
7.2.2 Bayesian Formulation . . . 13
7.2.3 Motion Model . . . 13
7.2.4 Measurement Model (Likelihood Field Model) . . . 14

7.2.5 Resampling . . . 17
7.3 Kidnapped Robot Problem . . . 18
7.3.1 Limitations and Failure Modes . . . 19

8 Navigation **19**
8.1 A^* . . . 19
8.2 Geometric Path Post-Processing and Trajectory Synthesis . . . 19
8.2.1 Path Pruning: String Pull . . . 19
8.2.2 C^1 Continous Spline Interpolation (Catmull Rom) . . . 20

9 Trajectory Following **21**
9.1 Linear Quadratic Regulator (LQR) . . . 21

10 Semantic Control **23**

11 Experiments **23**
11.1 Velocity and Position Control Performance . . . 23
11.2 Localization Performance . . . 23
11.3 Trajectory Following Performance . . . 23

12 Conclusion and Possible Improvements **23**

1 Introduction

This report presents the design, implementation, and testing of Paesano, a holonomic autonomous mobile robotics platform. Paesano is built to navigate complex and ambiguous environments using state estimation from an Extended Kalman Filter and a Monte Carlo localization algorithm, and to follow planned trajectories using semantic control. The hardware and software architecture are designed to be modular and scalable. This makes for the possibility of future additions such as higher quality sensors or more advanced control algorithms.

This project was created over the course of two semesters, with the first focused on hardware and firmware foundations, while the second focused on state estimation and control algorithms. The following sections will detail the design choices, implementation details, and certain testing metrics.

2 Hardware Architecture

2.1 Parts List

Prices shown below are approximate and may fluctuate over time based on vendor availability, shipping, and market conditions. Total estimated cost of listed components: $644.04.

Part	Cost (USD)
HiWonder Chassis Kit	$185.00 (approx.)
PCB Kit	$9.99
Raspberry Pi 5 w/ Power Supply and Active Cooler	$162.99
LiDAR	$74.90
Buck Converter (USB-C 5V 5A)	$9.99
22AWG Arduino Wires (Assorted)	$6.98
Power Switch	$7.99
Standoffs	$7.99
Heat Shrink	$7.99
5V Buck Converter	$8.99
5200mAh LiPo Battery	$35.99
LiPo Charger	$39.98
Raspberry Pi Pico	$14.59
XT60 Connectors	$10.99
XT60 1-to-2 Splitter	$12.88
BNO085	$24.95
Motor Drivers	$9.90
Wago Connectors	$11.95

Additional Required Tools.

- Assorted screwdrivers
- Soldering iron
- Multimeter
- Access to a 3D printer

2.2 System Overview

The system is built around an architecture that separates high-level processing from low-level and noisy motor control. The high-level compute module (Raspberry Pi 5 (RPI5) running Ubuntu LTS 24.04) handles state estimation, navigation, and high-level control/planning, while the motor control module (Raspberry Pi Pico (Pico), and TB6612 Motor Drivers) manages low-level actuation of motors. This separation allows for modularity and scalability, allowing future upgrades or repairs to parts without significant redesign of the entire system. Below (Figure 1) is a block diagram of the system architecture,

showing the flow of information and power between components. The high-level compute module receives sensor data, processes it for state estimation and navigation, and sends control commands to the motor control module, which then actuates the motors accordingly. This architecture also allows for real-time feedback and adjustments based on sensor inputs, ensuring robust performance in dynamic environments.

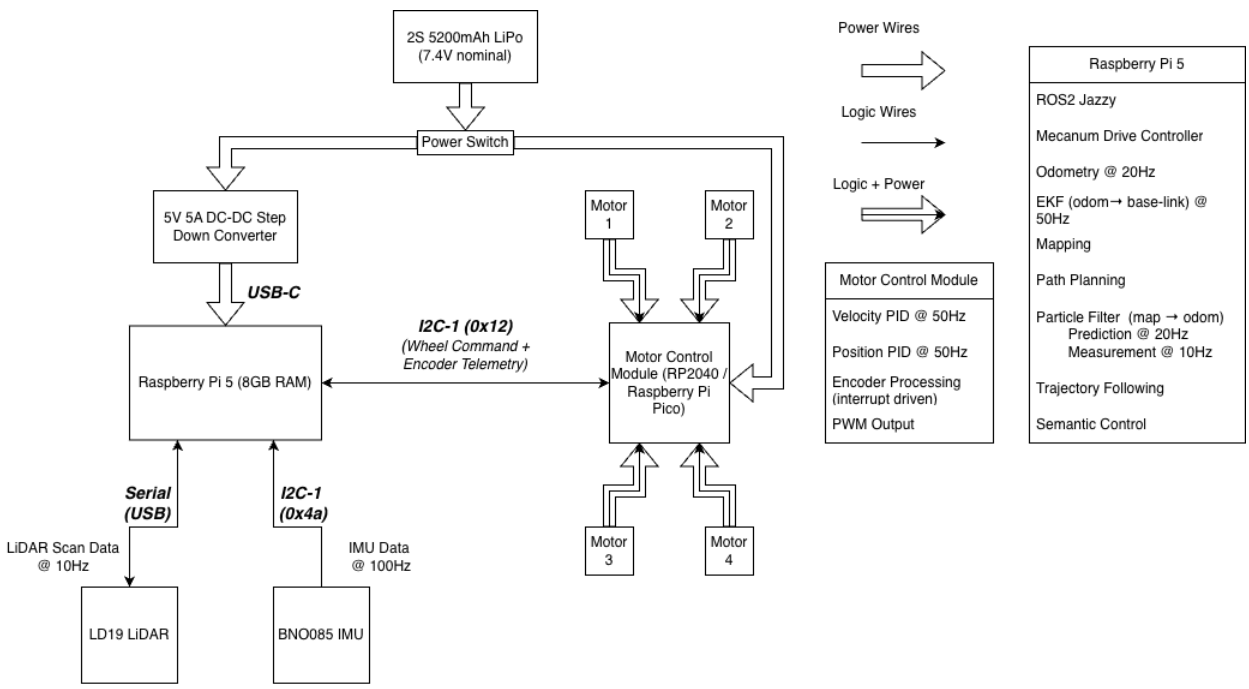


Figure 1: System architecture overview.

2.3 Power Architecture

The robot is powered by a 7.4 nominal voltage 2S 5200mAh LiPo battery. This battery directly powers the entire system, acting as the "heart" of the robot. This battery is chosen for its high energy density and ability to provide the necessary current for both the high-level compute module and the motor control module. A dedicated DC-DC converter is used to step down the voltage and amperage to appropriate levels for the RPI5, ensuring stable power delivery and preventing brownouts. The motors themselves are powered directly from the 7.4V supply, since they are designed to operate at this voltage.

The Pico is powered from a 5V line in the motor control module. This architecture ensures that transient current spikes from the motors don't affect the RPI5.

2.4 Motor Control Stack

The HiWonder chassis originally came with its own motor control module with a PID controller. However, the parts composing it weren't very powerful, and the ability to customize the firmware to fit the purposes of this project was nearly impossible. Instead, low-level motor control is handled by a dedicated motor control module built around the Pico and two TB6612 Dual Motor Drivers.

The motor control module performs real-time control of the 4 DC motors and executes closed-loop control at a fixed update rate of 50Hz. Offloading these control tasks to a microcontroller allows the motor control module to run with deterministic timing, as the Pico contains a real-time operating system, and ensuring the RPI5 uses its computational resources on higher level autonomy tasks. Below (Figure 2) is a schematic of the system.

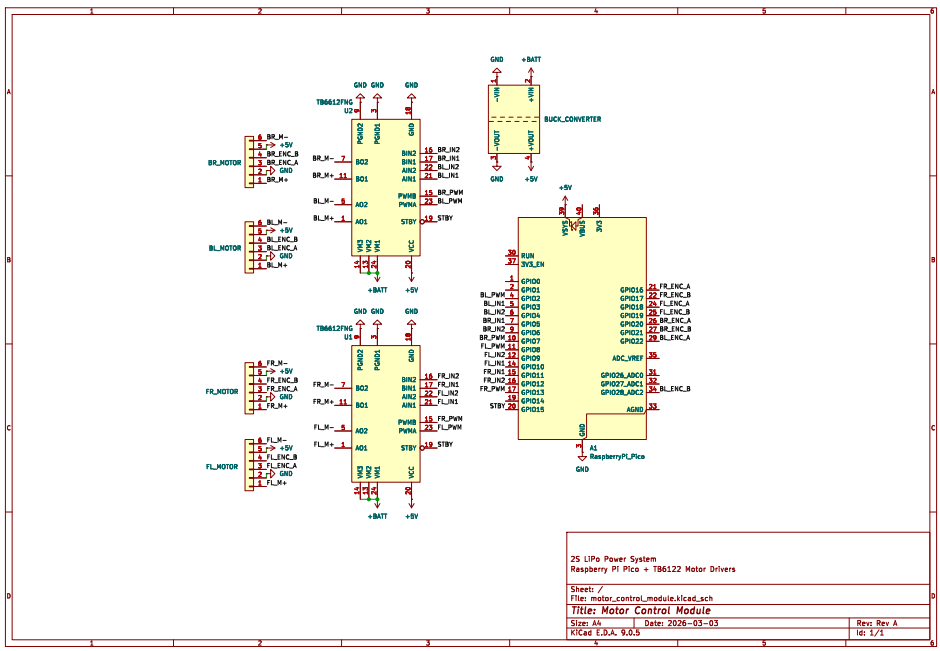


Figure 2: Motor Control Module.

2.5 Sensor Suite

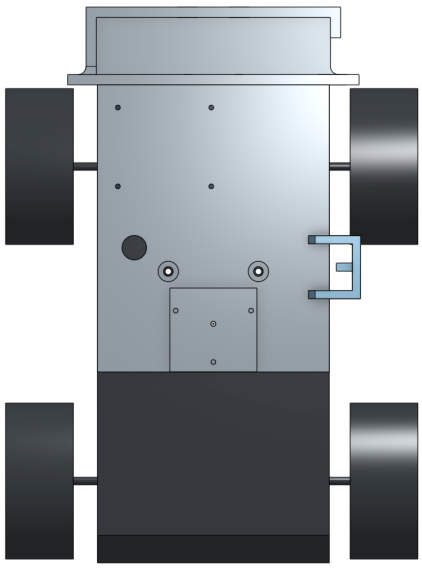
The robot uses a combination of wheel encoders, IMU, and LiDAR for perception and state estimation. A BNO085 IMU provides orientation and angular velocity measurements at approximately 100Hz. IMU data is fused with wheel odometry calculated using wheel encoders in an Extended Kalman Filter to produce a smooth local state estimate. A LD19 2D LiDAR sensor provides planar range measurements which is used for mapping and localization. The scan data is produced at approximately 10Hz.

- IMU Datasheet/Documentation: `https://www.adafruit.com/product/4754`

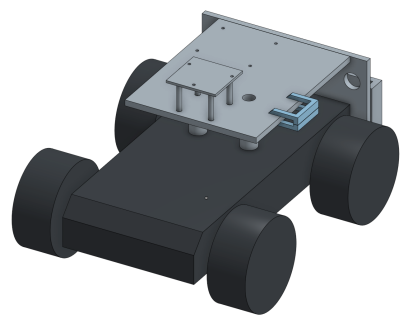
- LiDAR Datasheet/Documentation: `https://wiki.youyeetoo.com/en/Lidar/D300`

2.6 Mechanical Design

The chassis from HiWonder provided motors with encoders and a main chassis frame. However, it did not come with any intuitive way to mount the various sensors, batteries, and other processing units. To address this, several custom mounting parts were designed to integrate these systems into the platform. These parts were designed using OnShape and then printed on a Bambu Lab X1 Carbon in PLA.



(a) CAD Bird's Eye View



(b) CAD Isometric View

Figure 3: Mechanical CAD models of the platform and custom mounts.

The parts in gray and in blue are the ones that were custom designed and printed. The part in black is a model of the HiWonder chassis to make designing the custom parts easier. The parts in gray include a stand for the LiDAR, a mounting plate for the RPI5 and IMU, and a battery compartment in the back. The blue part is a power switch holder, and once printed it is to be super glued onto the gray mounting plate wherever desired.

2.7 Design Philosophy

The design philosophy of this robot is defined in 5 ideas:

- Modularity
- Simplicity
- Expandability
- Accessibility
- Cost Effectiveness

The robot was built in a way that makes it easy to debug and iterate on. Components are physically accessible and logically separated so that individual parts of the system can be tested or replaced without affecting the entire robot. For example, low-level motor control is handled independently from the high-level compute module, allowing each layer to be developed and debugged separately. Cost effectiveness

was also considered when selecting components, favoring reliable off-the-shelf parts that still allow room for future expansion. At the same time, simplicity was prioritized by avoiding unnecessary complexity and additional hardware when it was not required. This approach results in a system that is practical to develop on, easy to modify, and capable of supporting future improvements.

3 Firmware

All of the firmware for this project was written in C++ for its compatibility with the RP2040 microcontroller on the Pico. The firmware is responsible for the real-time motor control, encoder processing, and communication with the RPI5. Because motor control requires consistent timing and low latency, the firmware operates completely independent of the RPI5, which runs the high-level tasks.

3.1 Communication Protocol

Communication between the RPI5 and the Pico is implemented over I^2C, with the RPI5 acting as the bus master and the Pico configured as an I^2C slave at address `0x12`. The RPI5 reads and writes register-mapped data to exchange control commands and telemetry.

Table 2: I^2C register map between the RPI5 and Pico.

Name	Address	Pico Access	Purpose
`I2C_ADDR`	`0x12`	N/A	7-bit I^2C slave address for the Pico.
`REG_MOTOR_SPEEDS`	`51`	Read	Sets wheel/motor speed targets.
`REG_ENCODERS`	`60`	Write	Readback register for encoder counts or wheel feedback values.
`REG_PID_GAINS`	`70`	Read	Configuration register for PID gain parameters.
`REG_TELEM_FL`	`73`	Write	Telemetry register for front-left motor/channel data.

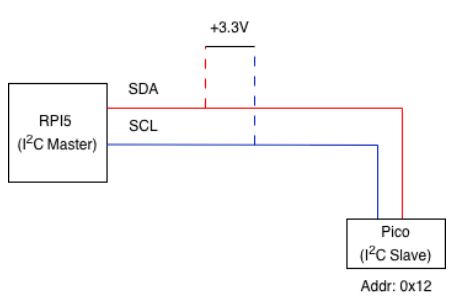


Figure 4: Schematic of the I^2C connection.

3.2 Control Loop Architecture

The main control loop of the firmware runs every 0.02s, or at 50Hz. Encoder measurements are used to estimate wheel velocities, which are fed into a cascaded control system. There are two possible modes: drive and hold. In

drive mode, the robot will move at the commanded velocity using a velocity PID with feedforward compensation. In hold mode (after the robot has been commanded to stop), a position controller generates velocity targets based on the error between the current position and the position at the time the robot was commanded to stop. These targets are then tracked by the velocity PID controller. The resulting PWM signals are sent to the TB6612 motor drivers.

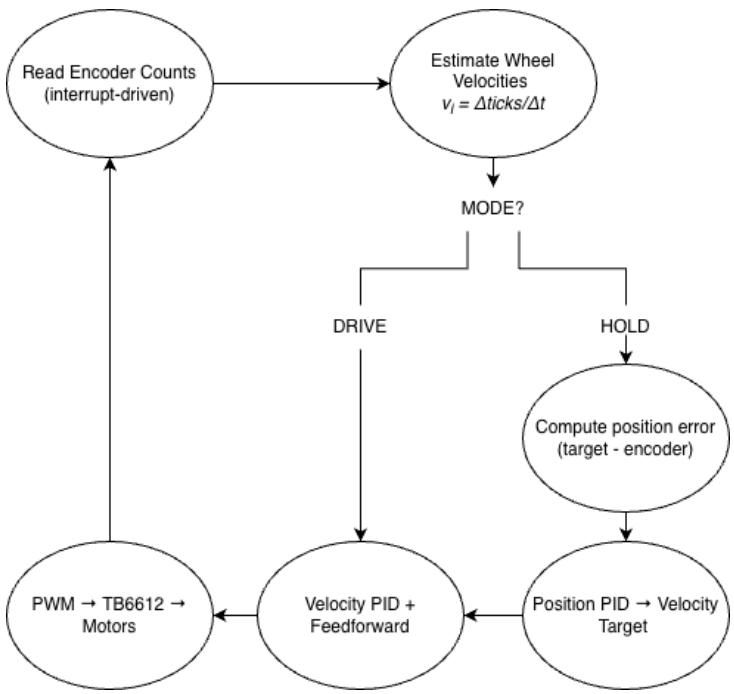


Figure 5: Control loop timing diagram.

3.3 Motor Control Algorithms

As aforementioned, motor velocity and position control is done using separate velocity and position PID controllers. Both controllers' integral term is capped when the control output saturates.

3.3.1 Velocity PID Control

$$e_v(t) = v_{\text{ref}}(t) - v(t)$$

$$u_v(t) = K_{p,v} e_v(t) + K_{i,v}\,\text{clip}\left(\int_0^t e_v(\tau)\,d\tau,\, -I_{\text{max},v},\, I_{\text{max},v}\right) + K_{d,v}\frac{de_v(t)}{dt} + K_{\text{ff}}\,v_{\text{ref}}(t)$$

3.3.2 Position PID Control

$$e_p(t) = p_{\text{ref}}(t) - p(t)$$

$$v_{\text{ref}}(t) = K_{p,p}e_p(t) + K_{i,p}\,\text{clip}\left(\int_0^t e_p(\tau)\,d\tau,\ -I_{\text{max},p},\ I_{\text{max},p}\right) + K_{d,p}\frac{de_p(t)}{dt}$$

4 Software Architecture

The software in the high-level compute module, running on the Raspberry Pi 5 (RPI5), is built using the Jazzy Jalisco distribution of ROS2 on Ubuntu 24.04 LTS. ROS2 provides a modular architecture that allows complex robotic systems such as this to be composed of smaller independent nodes, communicating over standardized topics. Using ROS2 provides several advantages for this system. First, it enables clear separation between the subsystems of perception, state estimation, planning, and control, where each system is its own node or series of nodes. Second, the publish-subscribe communication model used by ROS2 topics makes it simple to route sensor data and commands between nodes.

In this system, sensor measurements such as wheel encoder data, IMU readings, and LiDAR scans are published on dedicated topics and used by estimation nodes that fuse these measurements to produce a consistent state estimate of the robot. Higher level nodes for navigation and trajectory following use this state estimation. Robot commands are eventually published as velocity commands which are sent to the motor control module to convert them into actual motor actuation commands.

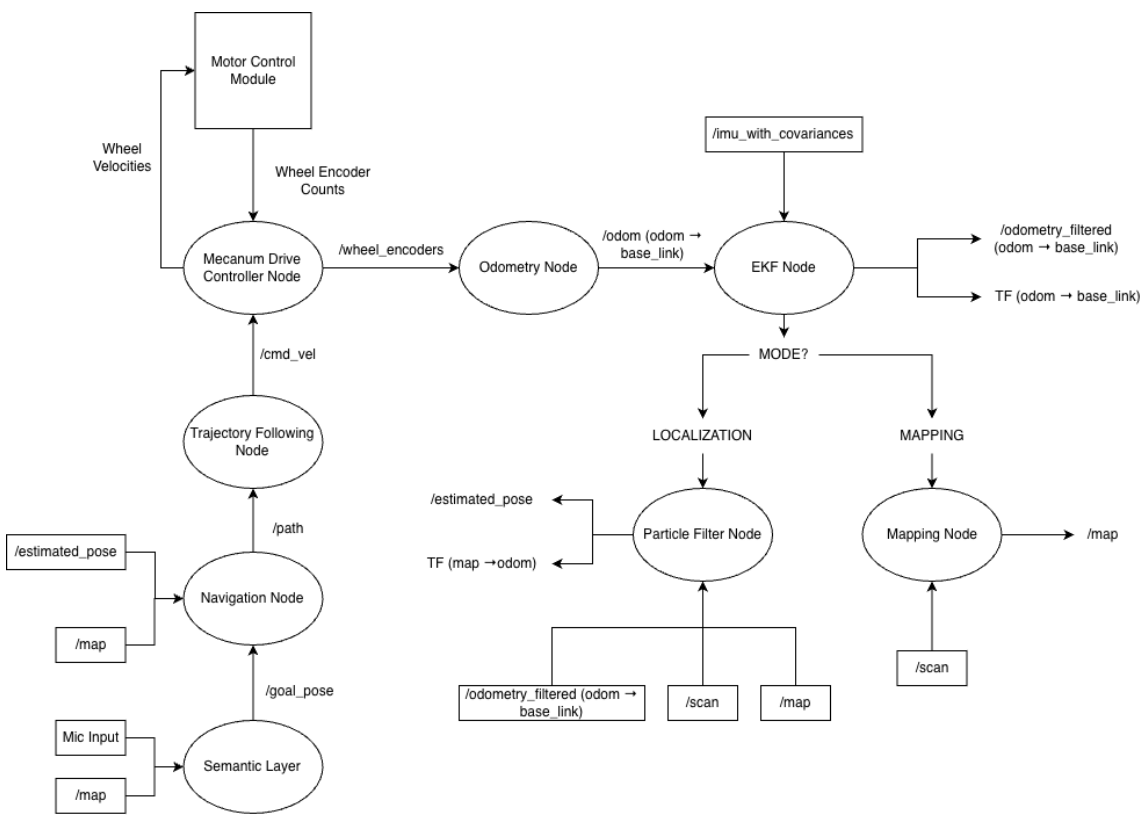


Figure 6: Software architecture overview.

The TF frame tree defines the relationships between the robot base (base_link), odometry frame (odom), map frame (map), and onboard sensors (base_laser, imu_link). These transforms are used by localization, navigation, and control nodes so pose estimates and commands remain consistent across the full software stack.

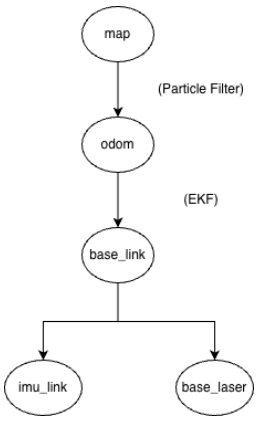


Figure 7: TF frame relationships used by the system.

5 Kinematics

To achieve holonomic motion, the robot uses a mecanum wheel drivetrain. Alternative systems such as swerve drive offer superior traction, efficiency, and high speed performance, but it also requires additional steering actuators and more complex mechanical control systems. For a system of this size, the increased hardware and software complexity isn't justifiable.

Mecanum wheels provide omnidirectional movement using only four actuators, creating a simple yet fully holonomic system. It is for this reason that mecanum drive was selected for this robot.

$$\begin{bmatrix} v_{\mathrm{FL}} \\ v_{\mathrm{FR}} \\ v_{\mathrm{BL}} \\ v_{\mathrm{BR}} \end{bmatrix} = \begin{bmatrix} 1 & -1 & -1 \\ 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} v_y \\ v_x \\ \omega_z \end{bmatrix}$$

$$\mathbf{v}_{\mathrm{wheel}} \leftarrow \frac{1}{\max(1, \max_i |v_i|)} \mathbf{v}_{\mathrm{wheel}}$$

This system uses a simplified approach to kinematics in which wheel PWM commands are generated directly from the commanded velocity input rather than from a full geometry-based inverse kinematics model using parameters such as wheel radius and horizontal/vertical wheel distances. This approach was chosen for two practical reasons. First, it guarantees bounded wheel commands under all requested motions, which improves saturation behavior and stability in the motor control module. Second, it avoids introducing sensitivity to geometric calibration errors in parameters such as wheel radius and chassis dimensions during early development.

6 Odometry

Wheel odometry provides a local estimate of robot motion by integrating measured wheel velocities over time. This estimate is subject to drift due to wheel slip, friction, noise, and other factors, but it provides a high frequency state estimate that becomes useful for localization later on. It is calculated using encoder measurements from each of the four drive actuators. Each one of them

provides incremental encoder counts (approx. 2882 per revolution) which are used to estimate wheel angular velocity. From these measurements the robot body velocity is estimated and then integrated to produce a pose estimate in the odom frame.

Let Δn_i denote the change in encoder ticks for wheel i over a time interval Δt. Using the known ticks-per-revolution constant of the encoders, the wheel angular velocity can be computed as

$$\omega_i = \frac{2\pi}{2882}\frac{\Delta n_i}{\Delta t}$$

The velocity of the robot itself is then estimated using the mecanum wheel forward kinematics. This matrix maps the individual wheel velocities to the robot translational velocity and angular velocity.

$$\begin{bmatrix} v_{x,\text{body}} \\ v_{y,\text{body}} \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ -\frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{1}{4} \\ -\frac{1}{4(L+W)} & \frac{1}{4(L+W)} & -\frac{1}{4(L+W)} & \frac{1}{4(L+W)} \end{bmatrix} \begin{bmatrix} \omega_{\text{FL}} \\ \omega_{\text{FR}} \\ \omega_{\text{BL}} \\ \omega_{\text{BR}} \end{bmatrix}$$

We then convert this robot velocity into the odom frame using a rotation matrix where theta is the heading of the robot, allowing the velocity to be represented in the odom frame:

$$\begin{bmatrix} v_{x,\text{odom}} \\ v_{y,\text{odom}} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} v_{x,\text{body}} \\ v_{y,\text{body}} \end{bmatrix}$$

The robot pose is then updated by integrating the body velocity in the odom frame:

$$x_{t+1} = x_t + v_{x,\text{odom}}\Delta t$$

$$y_{t+1} = y_t + v_{y,\text{odom}}\Delta t$$

$$\theta_{t+1} = \theta_t + \omega\Delta t$$

7 State Estimation

State estimation in this system is split into two connected layers: local fusion and global correction. The EKF fuses wheel odometry and IMU data to produce a smooth local estimate in the odom frame, while Monte Carlo Localization (MCL) uses LiDAR and the map to correct global drift. Operationally, the EKF provides `odom` → `base_link` and MCL provides `map` → `odom`, giving:

$${}^{map}T_{base_link} = {}^{map}T_{odom}\,{}^{odom}T_{base_link}$$

7.1 Extended Kalman Filter (EKF)

For the purposes of this project, the EKF provided by the *robot_localization* package was used for simplicity of integration (`https://wiki.ros.org/robot_localization`).

7.1.1 Purpose in the System

The purpose of the EKF is to generate a high-frequency local state estimate (x-position, y-position, v_x, v_y, heading θ, and heading rate $\dot{\theta}$) by fusing noisy sensor data from wheel odometry and the IMU, using covariance matrices to represent the relative uncertainty of each measurement. This local estimate is used as the motion model for MCL. The underlying prediction/update equations follow the standard *robot_localization* implementation.

7.1.2 State Vector

The EKF estimates a six-dimensional state representing the robot's planar pose and velocity.

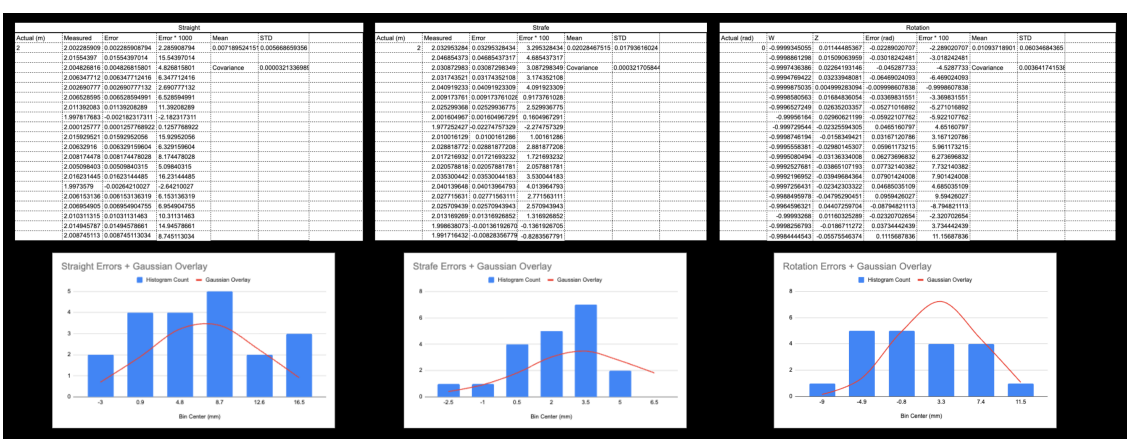
$$\mathbf{x} = [x, y, v_x, v_y, \theta, \dot{\theta}]^T$$

Odometry x, y, v_x, v_y, θ, $\dot{\theta}$ and IMU θ, $\dot{\theta}$ were fused to get this state. However, since odometry θ, $\dot{\theta}$ is extremely unreliable, covariances were tuned so that odometry practically owns x, y, v_x, v_y and the IMU owns θ and $\dot{\theta}$.

7.1.3 Covariance Tuning

The covariance values used in the state estimation pipeline were informed by experimental odometry measurements. While the experiment was conducted in ideal conditions, the results reveal important relative characteristics of the drivetrain noise. The experiment shows that forward motion produces the lowest odometry error, while lateral (strafing) motion exhibits significantly higher uncertainty due to the inherent slip characteristics of mecanum wheels. Rotational motion introduces the largest error, as heading estimation accumulates encoder noise and wheel slip. Across the trials, this is the relationship between each covariance:

$$\sigma_y^2 \approx 10\sigma_x^2, \qquad \sigma_\theta^2 \approx 10\sigma_y^2$$



Straight

Actual (m)	Measured	Error	Error * 1000	Mean	STD
2	2.002285909	0.002285908794	2.285908794	0.007189524151	0.005688659356
	2.01554397	0.015543970014	15.54397014		
	2.004826816	0.004826815801	4.826815801	Covariance	0.000032133686
	2.006347712	0.006347712416	6.347712416		
	2.002690777	0.002690777132	2.690777132		
	2.006528595	0.006528594991	6.528594991		
	2.011392083	0.011392082889	11.39208289		
	1.997817683	-0.002182317311	-2.182317311		
	2.000125777	0.000125776922	0.125776922		
	2.015929521	0.015929520056	15.92952056		
	2.00632516	0.006329159604	6.329159604		
	2.008174478	0.008174478028	8.174478028		
	2.005098403	0.005098403015	5.09840315		
	2.016231445	0.016231444485	16.23144485		
	1.9973579	-0.002642103027	-2.642103027		
	2.006153136	0.006153136319	6.153136319		
	2.006954905	0.006954904755	6.954904755		
	2.010311315	0.010311131463	10.31131463		
	2.014945767	0.014945678661	14.94578661		
	2.008745113	0.008745113034	8.745113034		

Strafe

Actual (m)	Measured	Error	Error * 100	Mean	STD
2	2.032953284	0.0329532843 4	3.295328434	0.020284675151	0.01793616024
	2.046854373	0.0468543737	4.685437317		
	2.030872983	0.0308729834 9	3.087298349	Covariance	0.00032170584
	2.031743521	0.0317435210 8	3.174352108		
	2.040919233	0.0409192330 9	4.091923309		
	2.009173761	0.0091737610 2	0.917376102 8		
	2.025299368	0.0252993675	2.529936775		
	2.001604987	0.0016049872 91	0.160498729 1		
	1.977252427	-0.0227475732 9	-2.274757329		
	2.010016129	0.0100161286	1.00161286		
	2.028818772	0.0288187720 8	2.881877208		
	2.017216932	0.0172169323 2	1.721693232		
	2.020578818	0.0205788178 1	2.057881781		
	2.035300442	0.0353004418 3	3.530044183		
	2.040139648	0.0401396479 3	4.013964793		
	2.027715631	0.0277156311 1	2.771563111		
	2.025709439	0.0257094394 3	2.570943943		
	2.013169269	0.0131692685 2	1.316926852		
	1.998638073	-0.0013619267 6	-0.136192676 5		
	1.991716432	-0.0082835677 8	-0.828356779 1		

Rotation

Actual (rad)	W	Z	Error (rad)	Error * 100	Mean	STD
	0 -0.9999345055	0.0114448367	-0.0228902070 7	-2.289020707	0.01093718901	0.06034684365
	-0.9998861298	0.0150906395 9	-0.0301824248 1	-3.018242481		
	-0.9997436386	0.0226419314 6	-0.045287733	-4.5287733	Covariance	0.00364174153 8
	-0.9994769422	0.0323394081	-0.0649924093	-6.499924093		
	-0.9996875035	0.0049992839 4	-0.0099986078 38	-0.999860783 8		
	-0.9998580563	0.0168483605 4	-0.0336983155 1	-3.369831551		
	-0.9996527249	0.0263520335 7	-0.0527101689 2	-5.271016892		
	-0.9995614	0.0296062119 9	-0.0592210776 2	-5.922107762		
	-0.9997295444	-0.0232559430 5	0.0465160797	4.65160797		
	-0.9998746194	-0.0158349421	0.0316712078 6	3.167120786		
	-0.9995558381	-0.0298014530 7	0.0596117321 5	5.961173215		
	-0.9995080494	-0.0313634008	0.0627369683 2	6.273696832		
	-0.9992527681	-0.0386510719 3	0.0773214038 2	7.732140382		
	-0.9992199552	-0.0394984364	0.0790142400 8	7.901424008		
	-0.9997256431	-0.0234203032 2	0.0468503510 9	4.685035109		
	-0.9988495978	-0.0479529045 1	0.0959426027	9.59426027		
	-0.9964596321	0.0440725970 4	-0.0879482111 3	-8.794821113		
	-0.99993268	0.0116032528 9	-0.0232070265 4	-2.320702654		
	-0.9998256783	-0.0198711272	0.0373442439	3.734442439		
	-0.9984444543	-0.0557554637 4	0.1115687836	11.15687836		

Figure 8: EKF covariance tuning reference.

These relationships were used to guide the structure of the odometry covariance matrix used by the filter. Because the experiments were conducted under controlled conditions, the resulting covariance values represent optimistic estimates of drivetrain noise. The exact covariance values were

therefore tuned empirically to maintain stable filter behavior across different operating environments, while preserving the experimentally observed ratios between forward, lateral, and rotational uncertainty.

IMU covariance was informed from the datasheet noise standard deviation. It was then converted to variance, then tuned experimentally.

7.2 Monte Carlo Localization (MCL)

7.2.1 Purpose in the System

Monte Carlo Localization was chosen because it is robust to non-Gaussian uncertainty (wheel slip, LiDAR noise, sudden disturbances), multi-modal pose hypotheses (considering all possible robot poses until new data arrives to disambiguate it), and occasional odometry drift. It also works well with the LiDAR likelihood-field measurement model used in this system, and can recover from poor initial pose estimates through resampling and re-weighting.

7.2.2 Bayesian Formulation

Monte Carlo Localization is derived from the recursive Bayes filter. The belief over the robot state is updated using both the motion model (control input) and the sensor model (measurement likelihood).

$$bel(w_t) = \eta\, p(z_t \mid w_t) \int p(w_t \mid w_{t-1}, u_t)\, bel(w_{t-1})\, dw_{t-1} \tag{1}$$

where

$$w_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$$

represents the robot pose at time t. The constant η is a normalization factor ensuring that the belief distribution integrates to one.

7.2.3 Motion Model

The motion model uses odometry increments to propagate each particle. First, the odometry displacement is rotated into the robot body frame:

$$\begin{bmatrix} \Delta x_{body} \\ \Delta y_{body} \end{bmatrix} = \begin{bmatrix} \cos(-\theta_{odom,t-1}) & \sin(-\theta_{odom,t-1}) \\ -\sin(-\theta_{odom,t-1}) & \cos(-\theta_{odom,t-1}) \end{bmatrix} \begin{bmatrix} \Delta x_{odom} \\ \Delta y_{odom} \end{bmatrix} \tag{2}$$

where $\theta_{odom,t-1}$ is the odometry heading at time $t-1$.

Next, that body-frame displacement is rotated into the frame of the particle:

$$\begin{bmatrix} \Delta x_{part} \\ \Delta y_{part} \end{bmatrix} = \begin{bmatrix} \cos(\theta_{part,t-1}) & \sin(\theta_{part,t-1}) \\ -\sin(\theta_{part,t-1}) & \cos(\theta_{part,t-1}) \end{bmatrix} \begin{bmatrix} \Delta x_{body} \\ \Delta y_{body} \end{bmatrix} \tag{3}$$

The particle state is then updated as

$$x_t = x_{t-1} + \Delta x_{part} \tag{4}$$
$$y_t = y_{t-1} + \Delta y_{part} \tag{5}$$
$$\theta_t = \theta_{t-1} + \Delta \theta_{odom} \tag{6}$$

To model uncertainty in the propagated particle state, the motion model is represented as a 3D Gaussian with covariance:

$$\Sigma_{w_t} = \begin{bmatrix} (x_{\text{noise}} \cdot translation)^2 & 0 & 0 \\ 0 & (y_{\text{noise}} \cdot translation)^2 & 0 \\ 0 & 0 & (\theta_{\text{noise}} \cdot rotation)^2 \end{bmatrix}$$

The probability density for the predicted particle pose is then

$$p(w_t) \propto \exp\left(-\frac{1}{2}(w_t - \mu)^T \Sigma^{-1} (w_t - \mu)\right) \tag{7}$$

where

$$\mu = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$$

is the mean predicted pose.

A sampled particle state is therefore generated as

$$\hat{w}_t = \mu + \mathcal{N}(0, \Sigma) \tag{8}$$

7.2.4 Measurement Model (Likelihood Field Model)

A likelihood field model works by computing probability of a particle being correct by projecting a beam from the robot onto each particle and getting the distance to the nearest known obstacle.

We represent this probability using a mixture model with Gaussian + uniform distributions:

Z_{hit} = probability that a reading was generated by a correct geometrical interaction with the map (Gaussian)

Z_{rand} = probability that a reading was generated by random noise or messy geometry (uniform distribution)

Z_{hit} and Z_{rand} must sum to 1.0.

$$P_{hit} = Z_{hit} \cdot \mathcal{N}(d_i; 0, \sigma^2)$$

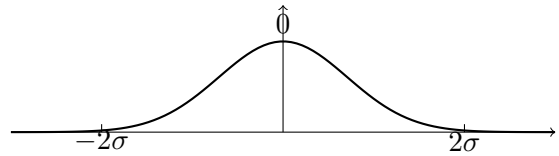


Figure 9: Gaussian likelihood model used for P_{hit} centered at the expected obstacle distance.

$$Z_{max} = \text{max range of lidar}$$

$$P_{rand} = Z_{rand} \cdot \frac{1}{Z_{max}}$$

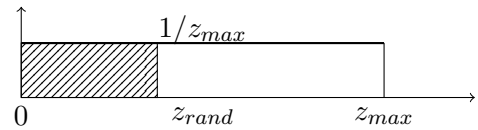


Figure 10: Uniform random measurement model used for P_{rand}.

We use a mixture model for robustness to incorrect sensor readings. For example, if a reading is 5m distance from the nearest obstacle, its Gaussian probability in log space would be:

$$\ln\left(\exp\left(\frac{-d^2}{2\sigma^2}\right)\right) = \ln(\exp(-1250)) = \ln(0)$$

since $\exp(-1250)$ underflows to 0.

$\ln(0)$ is undefined. This breaks the MCL update. By adding some probability that a reading wasn't Gaussian but part of a random uniform distribution between 0 and Z_{max} we prevent this.

So now,

$$P(z_i|x) = P_{hit} + P_{rand}$$

Where z_i is one beam and x is the particle. For a full scan containing k beams, the total likelihood is computed by

$$P(Z|x) = \prod_{i=1}^{k} P(z_i|x)$$

However, with each $P(z_i|x)$ being less than 1, $P(Z|x)$ will underflow quickly.

To avoid this we calculate $P(Z|x)$ in log space.

$$\ln(P(Z|x)) = \sum_{i=1}^{k} \ln(P(z_i|x))$$

In log space multiplication becomes addition and addition of log probabilities is more stable than multiplication of probabilities ≤ 1.

Since $P(z_i|x) = P_{hit} + P_{rand}$ we must compute

$$\ln(P_{hit} + P_{rand})$$

Mixture models involve a sum of two likelihood components and the overall scan likelihood is accumulated in log space. Therefore we compute the logarithms of each component separately. The sum is then evaluated using the log-sum-exp identity to ensure numerical stability.

Let

$$a = \ln(P_{hit})$$

$$b = \ln(P_{rand})$$

$$\ln(P_{hit} + P_{rand}) = \ln(e^a + e^b) = m + \ln(e^{a-m} + e^{b-m})$$

where

$$m = \max(a, b)$$

Now to calculate $\ln(P(Z|x))$ we compute

$$\ln(P(Z|x)) = \sum_{i=1}^{k} \left[m + \ln(e^{a-m} + e^{b-m}) \right]$$

For each particle we compute this. Now the vector of log weights for each particle is in log values and not normalized, so they are not probabilities. We must perform a log space normalization.

Let

$$m = \max_i \log(w_i)$$

where w_i is the vector of each particle's log weight.

Then define

$$\tilde{w}_i = w_i \cdot \exp(\log w_i - m)$$

Subtracting the maximum log weight shifts all values so the largest exponent is 0, preventing overflow.

Now we normalize

$$w_i = \frac{\tilde{w}_i}{\sum_j \tilde{w}_j}$$

so that

$$\sum_i w_i = 1$$

7.2.5 Resampling

Resampling particles is necessary because it reduces the problem of degeneracy where most particles get near-zero weight. This process:

- removes low-weight particles
- duplicates high-weight particles
- keeps the particle set focused on likely states
- improves estimation quality for the same compute budget

Resampling is triggered by calculating the effective sample size of all the particles, which in this implementation means how many particles are contributing meaningful weight to the state estimate. The equation for this is:

$$N_eff = \frac{1}{\sum_{i=1}^{N} w_i^2}$$

Where N is the total number of particles and w_i is the weight of each particle. An effective sample size closer to N means that most of the particles contribute meaningfully to the state estimate. An effective sample size closer to 1 means only a few particles dominate the estimate. We resample the particles when $N_eff < 0.5 * N$.

In this implementation, systematic resampling is done with a cumulative distribution (CDF) over normalized weights. This method achieves the 4 goals that we aforementioned. It removes low-weight particles because a low-weight particle will have a low interval in the CDF, making it less likely to be selected. It duplicates high-weight particles because a high-weight particle has a high interval in the CDF, making it more likely to be selected. Because it duplicates high weight particles it keeps the particle set focused on likely states, and only focusing on likely states improves estimation quality for the same compute budget. For a 10-particle example:

$$\mathbf{w} = [0.05, 0.10, 0.02, 0.18, 0.08, 0.07, 0.20, 0.06, 0.14, 0.10]$$

$$\mathbf{CDF} = [0.05, 0.15, 0.17, 0.35, 0.43, 0.50, 0.70, 0.76, 0.90, 1.00]$$

$$N = 10, \qquad \text{step} = \frac{1}{N} = 0.1, \qquad r \sim \mathcal{U}\left(0, \frac{1}{N}\right)$$

For this example, we use $r = 0.04$

This random offset r prevents the algorithm from repeatedly hitting the same CDF intervals. The systematic samples are:

$$u_i = r + (i-1) \cdot step \Rightarrow [0.04, 0.14, 0.24, 0.34, 0.44, 0.54, 0.64, 0.74, 0.84, 0.94]$$

For each u_i, pick the first CDF bin where $C_j \geq u_i$:

$$[0, 1, 3, 3, 5, 6, 6, 7, 8, 9]$$

These are the source particle indices copied into the resampled set, then all new weights are reset to $1/N$.

After resampling, we then add some Gaussian noise to the state of each particle, since the CDF may cause many particles to be exactly the same, causing particle impoverishment. Adding this noise spreads duplicated particles slightly and restores local diversity.

7.3 Kidnapped Robot Problem

To address the kidnapped robot problem, an adaptive random particle injection strategy was implemented

Given a set of particle log-likelihoods $\ell_i = \log p(z \mid x_i)$, the mean likelihood across particles is computed in a numerically stable manner using the log-sum-exp trick:

$$\log \sum_i e^{\ell_i} = \ell_{\max} + \log \sum_i e^{\ell_i - \ell_{\max}}$$

The average likelihood is then:

$$\log \bar{p} = \log \left(\frac{1}{N} \sum_i e^{\ell_i} \right) = \ell_{\max} + \log \sum_i e^{\ell_i - \ell_{\max}} - \log N$$

To normalize for the number of laser beams B, a scan quality metric is defined as:

$$q = \exp \left(\frac{\log \bar{p}}{B} \right)$$

Two exponential moving averages are maintained:

$$w_{\text{fast}} \leftarrow w_{\text{fast}} + \alpha_{\text{fast}}(q - w_{\text{fast}})$$
$$w_{\text{slow}} \leftarrow w_{\text{slow}} + \alpha_{\text{slow}}(q - w_{\text{slow}})$$

The ratio between these averages is used to detect localization degradation:

$$r = \frac{w_{\text{fast}}}{w_{\text{slow}}}$$

Using the ratio of fast and slow averages is more robust than reacting to the instantaneous confidence alone, since scan quality can fluctuate momentarily due to occlusion, dynamic obstacles, or local map ambiguity. The two time scales therefore detect a sustained drop relative to the recent baseline, which is a better indicator of the kidnapped robot condition.
An adaptive injection probability is then defined as:

$$p_{\text{adapt}} = \text{clamp}(1 - r, 0, 1)$$

This value is used to interpolate between a baseline and maximum random particle percentage:

$$\beta = \text{clamp}(k \cdot p_{\text{adapt}}, 0, 1)$$

$$P_{\text{random}} = P_{\text{base}} + (P_{\text{max}} - P_{\text{base}})\beta$$

The number of injected particles is:

$$N_{\text{inject}} = \frac{P_{\text{random}}}{100} \cdot N$$

Injected particles are sampled uniformly around the odometry estimate:

$$x \sim \mathcal{U}(x_{\text{odom}} \pm \Delta x), \quad y \sim \mathcal{U}(y_{\text{odom}} \pm \Delta y), \quad \theta \sim \mathcal{U}(\theta_{\text{odom}} \pm \Delta\theta)$$

To improve robustness, adaptive random injection is disabled when the robot is undergoing rapid rotation or executing planned navigation:

$$\text{allow} = (|\dot{\theta}| \leq \tau) \wedge (\neg\text{navigating})$$

This prevents spurious particle dispersion caused by scan distortion or intentional motion.

7.3.1 Limitations and Failure Modes

8 Navigation

8.1 A^*

Global path planning is performed using the A* algorithm on a 2D occupancy grid. Since the map is discretized into square cells, the raw output of A* is a sequence of adjacent grid coordinates rather than a continuous trajectory. This makes the initial path safe and graph-search optimal with respect to the grid cost model, but it also makes the output inherently jagged. Such a path is not ideal for a physical robot, especially one that will later be tracked by a continuous controller.

For this reason, the global planner is treated as the first stage of a path generation pipeline rather than the final trajectory itself. The A* search produces a collision-free sequence of cells, after which geometric post-processing is applied to reduce unnecessary waypoints and convert the discrete path into a smoother trajectory suitable for tracking.

8.2 Geometric Path Post-Processing and Trajectory Synthesis

8.2.1 Path Pruning: String Pull

The first post-processing step is a String Pulling algorithm. Its purpose is to prune redundant intermediate nodes introduced by the grid representation. Rather than forcing the robot to visit every cell transition selected by A*, the string-pull stage searches for the longest obstacle-free straight-line connections between waypoints.

This reduces the path to the minimum number of straight-line segments required to traverse the environment without colliding with obstacles. In practice, this removes much of the staircase-like structure produced by the grid and produces a cleaner geometric path before any smoothing is attempted.

8.2.2 C^1 Continous Spline Interpolation (Catmull Rom)

After pruning, the remaining path segments are smoothed using a Catmull-Rom spline. Catmull-Rom interpolation was selected because it produces a C^1 continuous path while still passing directly through the pruned waypoints. This makes it well suited for converting a piecewise linear path into a trajectory that is easier for the downstream controller to follow.

For four consecutive control points P_0, P_1, P_2, P_3, the interpolated position between P_1 and P_2 is given in matrix form by

$$\mathbf{p}(t) = \frac{1}{2}\begin{bmatrix}1 & t & t^2 & t^3\end{bmatrix}\begin{bmatrix}0 & 2 & 0 & 0\\ -1 & 0 & 1 & 0\\ 2 & -5 & 4 & -1\\ -1 & 3 & -3 & 1\end{bmatrix}\begin{bmatrix}\mathbf{P}_0\\ \mathbf{P}_1\\ \mathbf{P}_2\\ \mathbf{P}_3\end{bmatrix}, \qquad t \in [0,1]$$

Applying this component-wise gives

$$x(t) = \mathrm{CR}(x_0, x_1, x_2, x_3, t), \qquad y(t) = \mathrm{CR}(y_0, y_1, y_2, y_3, t)$$

where $\mathrm{CR}(\cdot)$ denotes the Catmull-Rom interpolation operator.

Initially, the spline was sampled by dividing each segment into a fixed set of parameter steps with $t \in [0,1]$. However, this introduced a fundamental problem: in spline mathematics, t is only a relative interpolation parameter and not a physical distance measure. A long segment and a short segment both span the same parameter range, so a fixed increment such as $\Delta t = 0.05$ covers much more physical distance on a long segment than on a short segment.

This caused the trajectory points to become unevenly spaced. In effect, the spline became "stretchy," with some parts of the path sampled sparsely and others densely. The LQR controller therefore received a fluctuating geometric error signal, which translated into inconsistent velocity behavior.

To correct this, the final trajectory is generated using arc-length parameterization. Instead of sampling the spline according to the interpolation parameter t, the path is re-sampled according to physical distance s traveled along the curve. For a parametric spline $\mathbf{p}(t) = [x(t), y(t)]^T$, the arc length from 0 to t is

$$s(t) = \int_0^t \left\| \frac{d\mathbf{p}(\tau)}{d\tau} \right\| d\tau = \int_0^t \sqrt{\left(\frac{dx}{d\tau}\right)^2 + \left(\frac{dy}{d\tau}\right)^2}\, d\tau$$

A numerical approximation is used to interpret this integral. The process is implemented in three stages:

1. **Dense modeling.** Each spline segment is first sampled at high resolution to create a dense approximation of the curve.
2. **Cumulative distance calculation.** The Euclidean distance between consecutive dense points is accumulated to compute the arc length at every sampled location.
3. **Uniform re-sampling.** The dense model is then re-sampled into trajectory points spaced at fixed 5 cm intervals.

If the dense samples are denoted $\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_N$, then the discrete cumulative arc length is approximated as

$$s_0 = 0, \qquad s_k = s_{k-1} + \|\mathbf{p}_k - \mathbf{p}_{k-1}\|, \quad k = 1, \ldots, N$$

Let the dense samples be denoted

$$\mathbf{q}_0, \mathbf{q}_1, \ldots, \mathbf{q}_N$$

with cumulative arc lengths

$$\ell_0 = 0, \qquad \ell_k = \ell_{k-1} + \|\mathbf{q}_k - \mathbf{q}_{k-1}\|, \quad k = 1, \ldots, N$$

so that the total path length is

$$L = \ell_N$$

The re-sampled trajectory is then defined at uniform spatial intervals

$$\Delta s = 0.05\,\mathrm{m}, \qquad s_m = m\,\Delta s, \quad m = 0, 1, \ldots, \left\lfloor \frac{L}{\Delta s} \right\rfloor$$

For each target distance s_m, choose the index k such that

$$\ell_k \leq s_m \leq \ell_{k+1}$$

and define the interpolation weight

$$\lambda_m = \frac{s_m - \ell_k}{\ell_{k+1} - \ell_k}$$

The uniformly spaced trajectory sample is then obtained by linear interpolation:

$$\hat{\mathbf{p}}_m = (1 - \lambda_m)\mathbf{q}_k + \lambda_m \mathbf{q}_{k+1}$$

The result is a trajectory whose points are uniformly spaced in physical distance regardless of the original segment lengths. This produces a far more consistent reference for the controller and significantly improves tracking smoothness by preventing the waypoint density from changing unpredictably along the path.

9 Trajectory Following

9.1 Linear Quadratic Regulator (LQR)

Trajectory following is performed using a Linear Quadratic Regulator (LQR). Since the platform is a holonomic Mecanum robot operating at relatively low speed, a discrete-time kinematic state-space model is sufficient for the controller design. The state is chosen as the planar robot pose

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix}$$

and the control input is chosen as the commanded body velocities

$$\mathbf{u}_k = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}.$$

The discrete kinematic update for each state component is

$$x_{k+1} = x_k + \dot{x}\,\Delta t, \qquad y_{k+1} = y_k + \dot{y}\,\Delta t, \qquad \theta_{k+1} = \theta_k + \dot{\theta}\,\Delta t$$

which can be written in matrix form as

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$$

with

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{I}, \qquad \mathbf{B} = \Delta t \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \Delta t\,\mathbf{I}.$$

The objective is not only to reduce tracking error, but to do so while avoiding unnecessarily aggressive actuation. This is expressed through the infinite-horizon quadratic cost

$$J = \sum_{k=0}^{\infty} \left(\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k \right)$$

where $\mathbf{Q}$ penalizes state error and $\mathbf{R}$ penalizes control effort. In the implementation, $\mathbf{Q}$ is chosen as a diagonal weighting matrix so that position and heading error can be penalized independently, while $\mathbf{R}$ is chosen as a scaled identity matrix to penalize large velocity commands.

For the infinite-horizon case, the steady-state optimal solution is determined by the Discrete Algebraic Riccati Equation (DARE):

$$\mathbf{P} = \mathbf{A}^T\mathbf{P}\mathbf{A} - \mathbf{A}^T\mathbf{P}\mathbf{B}\left(\mathbf{R} + \mathbf{B}^T\mathbf{P}\mathbf{B}\right)^{-1}\mathbf{B}^T\mathbf{P}\mathbf{A} + \mathbf{Q}.$$

The matrix $\mathbf{P}$ is the steady-state cost-to-go matrix. Although it is a constant matrix once solved, it encodes dynamic behavior because it is derived from the state-space model and therefore incorporates how present errors propagate through time. In this sense, $\mathbf{P}$ is a quadratic form that summarizes the long-term penalty of being in a given state.

The resulting minimum infinite-horizon cost from state $\mathbf{x}_k$ is

$$J_{\min} = \mathbf{x}_k^T \mathbf{P}\, \mathbf{x}_k.$$

Here, $\mathbf{x}_k$ denotes the state presented to the regulator at time step k. In this trajectory-tracking setting, it is best interpreted as the pose error state relative to the current reference sample, rather than the robot's absolute global pose.

Once $\mathbf{P}$ is known, the optimal feedback gain is

$$\mathbf{K} = \left(\mathbf{R} + \mathbf{B}^T\mathbf{P}\mathbf{B}\right)^{-1}\mathbf{B}^T\mathbf{P}\mathbf{A}.$$

The matrix $\mathbf{K}$ determines how aggressively the controller reacts to pose error. Larger effective gains lead to stronger correction, while smaller gains lead to smoother but less aggressive behavior. In practical terms, $\mathbf{K}$ is the matrix that maps the state error into a corrective velocity command.

In standard LQR notation, the optimal feedback law is written as

$$\mathbf{u}_{fb} = -\mathbf{K}\mathbf{x}_{err}.$$

In this project, the tracking error is defined as desired pose minus current pose and then transformed into the robot's local frame. With this sign convention, the implemented feedback term is equivalently applied as

$$\mathbf{u}_{fb} = \mathbf{K}\mathbf{e}.$$

A feedforward term is also added so that the robot does not merely react to error, but also maintains the nominal direction of motion along the path. The final control law is therefore

$$\mathbf{u}_{total} = \mathbf{u}_{fb} + \mathbf{u}_{ff}.$$

The feedback term drives the robot back toward the uniformly spaced spline trajectory points, while the feedforward term supplies the nominal motion along the current path segment. This combination allows the controller to both track the geometric trajectory and maintain smoother motion along the arc-length-parameterized path.

10 Semantic Control

11 Experiments

11.1 Velocity and Position Control Performance

11.2 Localization Performance

11.3 Trajectory Following Performance

12 Conclusion and Possible Improvements